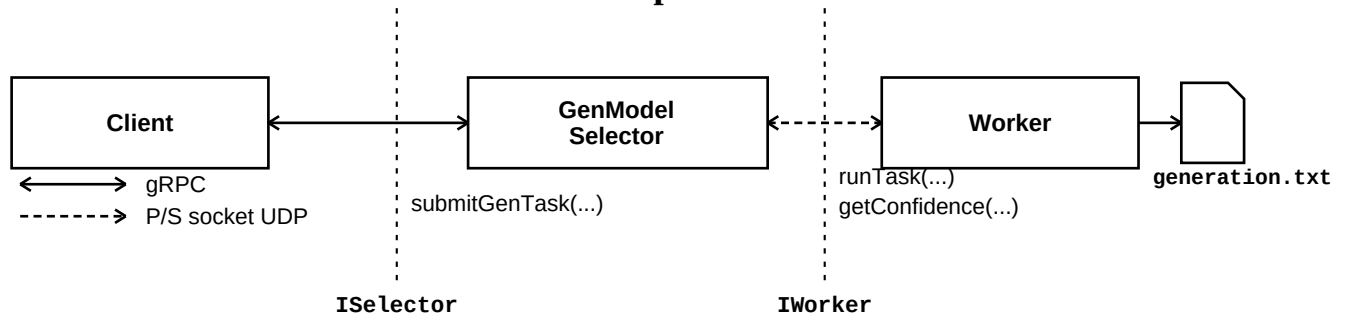


Testo della prova



Il candidato implementi un sistema distribuito in Python per la gestione automatica di task di *code generation* e *text generation* attraverso LLM, basato su gRPC Python e Socket UDP con Proxy/Skeleton. Il sistema è caratterizzato dai seguenti componenti.

Client. È un client gRPC che invia una richiesta di code generation al *GenModel Selector*, che è un server gRPC. L'invio della richiesta di code generation consiste nell'invocazione del metodo remoto *submitGenTask* specificato nell'interfaccia *ISector*. La comunicazione tra Client e GenModel Selector deve essere di tipo client unary / server streaming. La richiesta include i seguenti attributi: *taskId* (String) che identifica univocamente il task (e.g., task-1, task-2, etc.), *taskType* (String) che specifica il tipo di task (e.g., codeGen o textGen), *prompts* (array di String) un array di prompt, ognuno dei quali specifica una generazione diversa (e.g., prompt-1, prompt-2, etc.) e *numPrompt* (intero) che specifica il numero di prompt nell'array. Il Client genera 5 richieste e riceve per ognuna uno stream in risposta dal GenModel Selector. Lo stream conterrà un valore booleano per ogni prompt presente nella richiesta; ogni valore booleano indica se la generazione si è conclusa con una confidence che supera una certa soglia o meno. Il taskId della richiesta e i booleani ricevuti dal GenModel Selector per quella richiesta sono stampati a video. I parametri della richiesta possono essere generati casualmente.

GenModel Selector. È l'entità intermedia del sistema. Fornisce l'interfaccia gRPC *ISector* e il relativo metodo remoto *submitGenTask*. Il metodo riceve le richieste dal Client e per ogni richiesta: (i) verifica se il task di generazione è di tipo codeGen o textGen; (ii) estrae i prompt presenti nella richiesta; (iii) per ogni prompt invoca il metodo remoto **runTask** fornito dal Worker, specificando il prompt ed il modello (CodeT5, se il task è di tipo codeGen, o T5, se il task è di tipo textGen); (iv) attende l'esito di runTask, che restituisce un booleano pari a True (nel caso di generazione riuscita) o False (nel caso di problemi); (v) se l'esito di runTask è False, restituisce False nello stream in risposta al Client; se l'esito è True, invoca il metodo remoto **getConfidence** fornito dal Worker; (vi) attende l'esito di getConfidence (che restituisce un booleano che indica se la generazione ha avuto una confidence superiore a una certa soglia) e lo restituisce come parte dello stream in risposta al Client. L'interazione tra GenModel Selector e Worker avviene sfruttando il pattern Proxy/Skeleton.

Worker. Fornisce l'interfaccia *IWorker*, che definisce due metodi remoti *runTask* e *getConfidence*. Il GenModel Selector invoca runTask per ciascun prompt e, in caso di esito positivo, invoca successivamente getConfidence. Il metodo **runTask** simula l'esecuzione di un task di generazione con il modello indicato ed accetta come parametri: un prompt ed il modello da utilizzare. Ricevuta la richiesta, runTask scrive sul file **generation.txt** (in modalità append) sia il prompt che il modello ricevuti, e restituisce un booleano (selezionato in maniera casuale) per indicare l'esito della generazione. Il metodo **getConfidence** (che non ha parametri) simula il recupero della metrica di confidenza del modello (valore numerico tra 0 e 1 da generare in maniera casuale) e restituisce True se tale confidenza è superiore a 0,7 o False in caso contrario. Il Worker deve essere implementato in accordo con il pattern Proxy/Skeleton con socket UDP. Si utilizzi inoltre uno **Skeleton multithread** con schema per delega.